

Taxpreco Architecture and Alpha Deployment Report

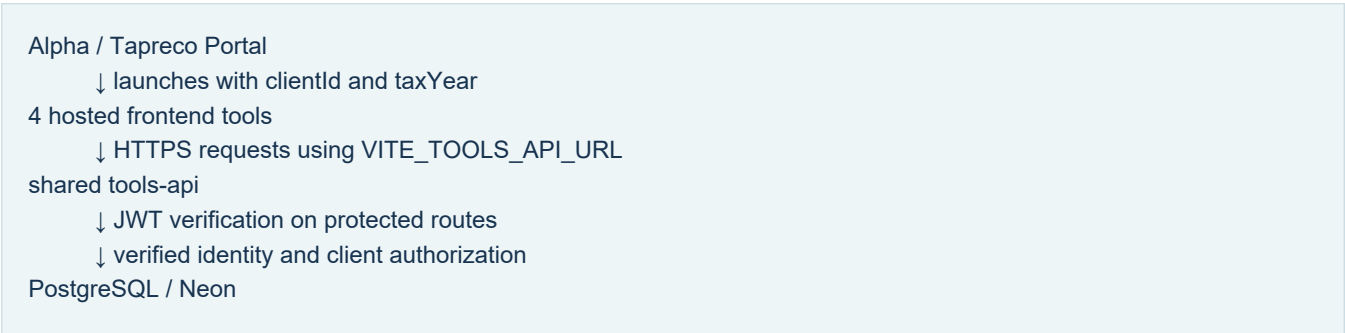
Date: 8 October 2026
Branch: feature/tools-integration
Audience: Taxpreco development team and Alpha integration team

Taxpreco contains four separate React frontend tools and a shared Express API with PostgreSQL integration for Mutual Fund. The proposed production structure is four Render Static Sites, one Render Web Service, and one shared PostgreSQL/Neon database. Real Alpha token delivery and client-data authorization must be completed before protected production use.

This report records the repository review without redesigning or changing application code. Source configuration was inspected; clean builds, running services, and live database save/load are **Needs confirmation**. The shell runner was unavailable during the review. Secret values are intentionally omitted.

A Current architecture

Shared service flow



Client authorization in this flow is a required production step, but is not currently enforced by the Mutual Fund routes. Public health and tool-catalog endpoints do not require JWT verification.

Repository packages

Location	Current implementation
apps/entity-comparison-web	React, Vite, TypeScript; dashboard and Entity Comparison demo
apps/mutual-fund-web	React, Vite, TypeScript; UI with authenticated database save/load requests
apps/estimated-tax-web	React, Vite, TypeScript; worksheet preview
apps/audit-risk-web	React, Vite, TypeScript; audit-risk UI placeholders
services/tools-api	Node.js, Express, CommonJS JavaScript; JWT verification and PostgreSQL queries
services/entity-comparison-api	Older Express/TypeScript backend; local mock identity and unfinished JWT verification

There are **four frontend packages and two backend packages in the repository**. The proposed deployment uses **one backend service: services/tools-api**. The older API remains in the checkout and is still used by root scripts.

Each package has its own package.json and package-lock.json. No npm workspaces are configured.

Root scripts and documentation

The root package.json still coordinates Entity Comparison and services/entity-comparison-api. Root npm run dev starts those two packages; npm run build builds only Entity Comparison. The server build/start/test commands target the older API. These commands do not start the four-tool shared-service architecture.

The root README.md and ARCHITECTURE_AUDIT.md describe the older Phase 1 setup, including no database. They are not a current description of tools-api.

Frontend connections

Frontend	Actual backend use
Entity Comparison	Imports src/api/client.ts. Calls GET /api/health, /api/tools, and /api/me. Reads VITE_TOOLS_API_URL. Its current identity call passes no token, so tools-api returns 401. Comparison edits remain in React memory.
Mutual Fund	Imports src/api.ts. Calls authenticated GET and POST /api/mutual-fund/sessions. Reads VITE_TOOLS_API_URL. Token is entered manually in a testing textarea.
Estimated Tax	Has src/api.ts using VITE_TOOLS_API_URL, but the current UI does not call it. No persistence integration found.
Audit Risk	Has src/api.ts using VITE_TOOLS_API_URL, but the current UI does not call it. No persistence integration found.

All four frontend API configurations fall back to http://localhost:4000 when the variable is absent. Entity Comparison also has a separate, unused src/api.ts helper; its active client is src/api/client.ts.

Shared API routes

Method and route	JWT required	Behavior
GET /health	No	Service liveness
GET /api/health	No	Service liveness
GET /api/tools	No	Catalog of four tools, all marked as demos
GET /api/context	Yes	Verified identity context
GET /api/me	Yes	Authenticated identity
GET /api/tools/:toolName/status	Yes	Status/context for one recognized tool
GET /api/mutual-fund/sessions	Yes	Loads rows for supplied clientId and taxYear
POST /api/mutual-fund/sessions	Yes	Inserts a Mutual Fund row

The status route recognizes entity-comparison, mutual-fund, estimated-tax, and audit-risk. It does not implement calculations or persistence for those tools.

Launch context

Every frontend has `src/launchContext.ts`. It reads `clientId` and `taxYear` from `window.location.search` using `URLSearchParams`, returning strings or null. These values are context, not proof of access permission.

Example launch query: `?clientId=client-123&taxYear=2026`

Tool	Current launch handling
Entity Comparison	Accepts a supplied <code>clientId</code> only if it matches a demo client; otherwise uses the first demo client. Year defaults to 2026.
Mutual Fund	Uses supplied values; defaults to <code>client-123</code> and 2026.
Estimated Tax	Initializes selectors; defaults to <code>sample-client</code> and 2026.
Audit Risk	Initializes selectors; defaults to <code>sample-client</code> and 2026.

Entity Comparison does not yet resolve arbitrary Alpha client IDs to real client records. Its default page is the dashboard; the `#entity-comparison` fragment opens the tool screen directly.

B Local development ports

Application	Configured local URL
Entity Comparison	<code>http://localhost:5173</code>
Mutual Fund	<code>http://localhost:5175</code>
Estimated Tax	<code>http://localhost:5176</code>
Audit Risk	<code>http://localhost:5177</code>
Shared tools-api	<code>http://localhost:4000</code>
Older entity-comparison-api	<code>http://localhost:3001</code>

These are configured ports, not confirmation of currently running processes. Vite can select a later available development port because strict-port enforcement is disabled or unspecified. Use the URL printed by Vite.

Entity Comparison preview uses port 4173 with strict-port enforcement. Existing browser tests configure frontend 5199 and the older API 3002.

Entity Comparison retains a development `/api` proxy to `localhost:3001`. Requests using an absolute `VITE_TOOLS_API_URL` bypass that proxy. An explicitly empty base in its active API client uses relative requests and therefore the proxy locally.

These local ports matter only on a developer machine. Production users access HTTPS URLs without `:5173` or `:4000`. Render's internal API listener port is separate from its public HTTPS URL.

C Production hosting plan

Create four Render Static Sites and one Render Node Web Service linked to `feature/tools-integration`. Keep each service rooted in its own package. Render resolves `build/start` commands and publish paths relative to the configured Root Directory. [Render monorepo documentation](#)

Static Sites

Site	Root Directory	Build Command	Publish Directory	Required environment variable	Public URL role
Entity Comparison	apps/entity-comparison-web	npm ci && npm run build	dist	VITE_TOOLS_API_URL	Alpha Entity Comparison destination
Mutual Fund	apps/mutual-fund-web	npm ci && npm run build	dist	VITE_TOOLS_API_URL	Alpha Mutual Fund destination
Estimated Tax	apps/estimated-tax-web	npm ci && npm run build	dist	VITE_TOOLS_API_URL	Alpha Estimated Tax destination
Audit Risk	apps/audit-risk-web	npm ci && npm run build	dist	VITE_TOOLS_API_URL	Alpha Audit Risk destination

Set the same shared API HTTPS origin as VITE_TOOLS_API_URL on all four sites. Each frontend build script runs tsc -b && vite build. No custom output directory is configured; expected publish output is dist.

Each Static Site receives a separate public Render URL and can use a custom domain. Actual assigned URLs are **Needs confirmation**. [Render Static Sites](#)

Backend Web Service

Setting	Value
Runtime	Node
Root Directory	services/tools-api
Build Command	npm ci
Start Command	npm start
Entrypoint	node src/server.js
Compilation	None; this service is JavaScript
Health Check Path	/health
Database	One shared PostgreSQL/Neon database
Public URL role	Shared HTTPS API for all four browser tools

Use Render's supplied PORT. The service reads process.env.PORT or falls back to 4000. It does not specify a loopback host in app.listen and has no configurable HOST setting. Render requires an all-interface listener and recommends its PORT, which defaults to 10000. Render terminates public HTTPS. [Render Web Services](#)

Use a consistent supported Node runtime. The frontend lockfiles record Vite's Node requirement as ^20.19.0 or >=22.12.0; the repository README recommends Node 24+. No package-level Node engine pin was found. Clean Linux deployment builds are **Needs confirmation**.

No Render configuration file or confirmed production deployment URL was found.

D Environment variables

Variables on each frontend Static Site

VITE_TOOLS_API_URL=https://<actual-tools-api-host>

Use the backend origin without /api because clients append their route paths. Avoid a trailing slash for consistent behavior across clients.

Vite substitutes VITE variables at build time, so changing this URL requires a frontend rebuild. These variables are visible in browser assets and must not contain secrets. [Vite environment documentation](#)

Variables in Alpha's portal

Alpha variable	Destination
VITE_ENTITY_COMPARISON_URL	Entity Comparison Static Site HTTPS URL
VITE_MUTUAL_FUND_URL	Mutual Fund Static Site HTTPS URL
VITE_ESTIMATED_TAX_URL	Estimated Tax Static Site HTTPS URL
VITE_AUDIT_RISK_URL	Audit Risk Static Site HTTPS URL

Alpha should use the matching destination and append URL-encoded clientId and taxYear. For Entity Comparison, append #entity-comparison when launching directly into the tool.

These four portal variables are not implemented in this repository; Alpha's portal implementation is **Needs confirmation**. They are separate from the VITE_TOOLS_API_URL used by each tool to call the backend. Do not replace them with VITE_APP_URL or VITE_APP_API_URL.

Variables on the shared backend

Variable	Current support
DATABASE_URL	Required private PostgreSQL connection setting. Missing value prevents startup.
JWT_SECRET	Read by jwt.verify. Required for the current verification approach; Alpha must confirm compatibility and supply settings securely.
PORT	Supported; use Render's supplied value.
NODE_ENV=production	Recommended environment setting; tools-api does not currently use it to enforce production safeguards.
CORS variable	None currently supported. The code uses unrestricted cors().

The local tools-api .env contains PORT, JWT_SECRET, and DATABASE_URL. PORT is 4000 and the database URL points to a Neon hostname. The .env.example includes PORT and JWT_SECRET but omits the required DATABASE_URL.

AUTH_MODE, HOST, SERVE_FRONTEND, TAPRECO_ALLOWED_ORIGIN, TAPRECO_JWT_ISSUER, TAPRECO_JWT_AUDIENCE, and TAPRECO_JWKS_URL are settings of the older entity-comparison-api. They do

not configure tools-api. Adding them to the shared service will not activate restricted CORS or issuer/audience/JWKS verification.

CORS requirements

The shared API should allow the four exact frontend origins, GET/POST requests, OPTIONS preflight, and Authorization/Content-Type headers. Include Alpha's portal origin only if the portal also directly calls the API; simply linking to tools does not require it.

The current cors() configuration allows browser access from any origin. An origin allowlist requires a later code change. CORS does not authenticate a user or authorize client-data access.

E JWT flow

```
Browser request with Authorization: Bearer <access-token>
  ↓ requireAuth
jwt.verify(token, JWT_SECRET)
  ↓ verified claims stored in req.user
attachContext
  ↓ userId from sub or userId
  ↓ clientId from clientId claim
  ↓ email from email claim
Protected route
```

Shared tools-api JWT verification exists. Real Alpha integration remains pending:

- Mutual Fund uses a manual JWT testing textarea and holds its token in React state.
- Entity Comparison currently passes no token to /api/me.
- Estimated Tax and Audit Risk do not currently make authenticated requests.
- tools-api has no explicit issuer/audience policy, algorithm allowlist, or JWKS implementation.
- A verified token is not required to contain a usable user ID.
- The older API's JWT mode remains an unfinished verification stub.

JWT must NOT be placed in the URL. URLs can appear in history, logs, and copied links. Launch URLs carry clientId and taxYear; JWT must use an agreed secure handoff and be sent in the Authorization header. Signing secrets/private keys must never enter frontend environment variables.

Navigation to a separately hosted frontend does not automatically transfer Alpha's token. Token delivery, expiry/refresh behavior, signing method, and required claims are **Needs confirmation** with Alpha.

F Database flow

The shared backend uses pg.Pool for one configured PostgreSQL database. Local configuration points to Neon. Live connectivity and production database selection are **Needs confirmation**.

Source file	Responsibility
services/tools-api/src/db/index.js	Connection pool and SELECT NOW() connection check
services/tools-api/src/db/init.js	CREATE TABLE IF NOT EXISTS at startup
services/tools-api/src/routes/mutualFund.js	Parameterized INSERT and SELECT queries

The `mutual_fund_sessions` table stores `id`, `user_id`, `client_id`, `tax_year`, `resident_state`, `fund_name`, `amount`, `percentage`, `state_exempt`, `state_taxable`, `created_at`, and `updated_at`. No persistence routes or tables were found for the other three tools.

Mutual Fund save and load

Save sends an authenticated POST with launch context and form data. The backend records the verified user ID and inserts a new row. Every save inserts another row; no update/upsert endpoint exists. The current UI submits `percentage=0`, `stateExempt=0`, and `stateTaxable=amount`, confirming persistence wiring rather than a completed fund calculation engine.

Load sends an authenticated GET with `clientId` and `taxYear`. The backend returns matching rows ordered by creation time, newest first. The frontend logs the result; **it does not restore saved values into the form**.

The user reports working PostgreSQL save/load. The source supports that integration, but a fresh live round trip is **Needs confirmation**.

Separation of responsibilities

JWT authenticates the user. The backend extracts verified identity and must determine which clients the user may access. Database queries then store or retrieve authorized records.

Current routes do not enforce this full separation. INSERT stores `req.context.userId` but accepts the submitted `clientId` without checking permission. SELECT filters only by `client_id` and `tax_year`, not user ID or authorized client membership. The extracted JWT `clientId` is not used to enforce access, and `user_id` is nullable.

Client-data authorization is a production blocker for real data. A valid token alone does not prevent access to another client's records.

The database pool currently uses SSL with `rejectUnauthorized: false`. Production certificate verification should be reviewed. Startup connection/table errors are logged while HTTP serving may continue, so `/health` does not prove database readiness.

G Alpha handoff summary

The following section can be sent to Alpha.

Deployment structure

We have four separate frontend tools: Entity Comparison, Mutual Fund, Estimated Tax, and Audit Risk. The deployment plan is four Render Static Sites, one shared Render Web Service for services/tools-api, and one shared PostgreSQL/Neon database. An older entity-comparison-api package remains in the repository but is outside this deployment plan.

Local and production addresses

Local ports are Entity Comparison 5173, Mutual Fund 5175, Estimated Tax 5176, Audit Risk 5177, and tools-api 4000. The older API uses 3001. Production will provide four frontend HTTPS URLs and one shared API HTTPS URL. Actual public URLs are **Needs confirmation** until deployment.

Alpha's portal should use VITE_ENTITY_COMPARISON_URL, VITE_MUTUAL_FUND_URL, VITE_ESTIMATED_TAX_URL, and VITE_AUDIT_RISK_URL. Each tool will use VITE_TOOLS_API_URL pointing to the same shared API. Do not use generic VITE_APP_URL or VITE_APP_API_URL names for these destinations.

Integration status

All tools parse clientId and taxYear from launch URLs. Entity Comparison currently accepts only demo client IDs. Shared-backend JWT verification exists using JWT_SECRET, but Alpha's token handoff is pending. JWT must travel through an agreed secure mechanism and the Authorization header, never in a URL.

Mutual Fund has PostgreSQL INSERT/SELECT integration; its Load action currently logs returned rows instead of restoring the form. The other tools do not yet have persistence integration. Real client-data access requires backend authorization enforcement and JWT/CORS hardening before release.

What Alpha still needs to provide

- Token handoff mechanism and expiry/refresh contract.
- Signing method and verification configuration: shared secret if applicable, or public-key/JWKS requirements.
- Expected issuer, audience, permitted algorithms, and user/client/tenant claims.
- Rules defining which clients each user may access.
- Portal origin and whether tools launch through navigation or embedding.
- Approved production client IDs and database ownership/access arrangements.
- Confirmation that the current demo/preview functionality is the intended initial hosting scope.

H Deployment blockers and checklist

Item	Status or required action
Client-data authorization	Blocker for real data. Enforce authorized client access on load and save.
Alpha JWT handoff	Blocker for authenticated portal use. Agree the contract and integrate token delivery.
JWT policy	Confirm signing method and enforce applicable issuer, audience, algorithms, and identity claims.
Frontend API URLs	Set VITE_TOOLS_API_URL on all four sites before building; otherwise browsers call localhost:4000.
Database configuration	Supply private DATABASE_URL and confirm production connectivity/table readiness.
Startup readiness	Database initialization errors do not stop HTTP serving; health is not a database readiness test.
Database TLS	Review rejectUnauthorized: false before production use.
CORS	Implement the approved frontend origin allowlist.
Builds and runtime	Clean builds of all four frontends and backend startup are Needs confirmation.
Root scripts and docs	Still describe/use the older API; use package-specific deployment commands.
Existing browser tests	Set the obsolete VITE_API_BASE_URL while the active client reads VITE_TOOLS_API_URL; align configuration.
Product functionality	Estimated Tax/Audit Risk are previews; Mutual Fund calculation and form restoration are incomplete. Confirm hosting scope.
Public URLs	Assign and verify four frontend HTTPS URLs and one API HTTPS URL.
Secret handling	Keep DATABASE_URL and JWT verification secrets on the backend only.

Git exclusions

The root .gitignore excludes node_modules, .env, and .env.* while allowing .env.example. The tools-api .gitignore additionally excludes .env and node_modules/.

The local services/tools-api/.env is absent from Git's index; example environment files are intentionally tracked. However, Git's index contains one dependency binary:

```
apps/estimated-tax-web/node_modules/@rolldown/binding-win32-x64-msvc/rolldown-binding.win32-x64-msvc.node
```

Ignore rules do not untrack existing files. Remove this dependency binary from tracking in a later authorized cleanup. Its remote/committed status is **Needs confirmation**. No application code or Git tracking was changed for this report.